

Failure and Recovery in Transaction Management

What happens when the power fails mid-transaction, and how the recovery manager restores a correct database using nothing but a log.

Agenda

1

Types of transaction failure

2

System failure vs. media failure

3

The recovery manager's job

4

Log-based recovery & write-ahead logging (WAL)

5

Undo and redo: what each reverses, and why both matter

6

Checkpointing, rollback, and rollforward

7

A full worked crash-recovery walkthrough

Types of Transaction Failure

Transaction Failure (Logical Error)

A constraint violation, a deadlock victim selection, a division by zero, or an explicit ROLLBACK. The DBMS and hardware are fine — only this one transaction needs undoing.

System Crash

The DBMS, OS, or machine stops unexpectedly — but the disk remains intact. Everything in volatile memory at the moment of the crash is lost.

Media (Disk) Failure

The storage device itself is damaged or destroyed. Unlike a system crash, the data ON DISK is no longer trustworthy or accessible at all.

System Failure vs. Media Failure

● System Failure

Volatile memory (RAM) is lost. Disk is intact. Recovery replays the log against the existing disk data — no backup needed.

● Media Failure

The disk itself is lost or corrupted. Recovery must restore from the most recent backup, then apply the log to bring it forward to the point of failure.

A backup you have never restored is not a backup. Teams routinely discover, only during an actual media failure, that their backup files were silently incomplete or never being written. The only way to trust a backup is to periodically actually restore it — on a test system — and confirm the result is complete and consistent.

The Recovery Manager's Job

The recovery manager restores the database to a correct, consistent state after any failure — concretely, it must guarantee that Atomicity and Durability both hold despite the failure.

● Atomicity, guaranteed

Every transaction that had NOT committed must show none of its effects at all, exactly as if it had never run.

● Durability, guaranteed

Every transaction that HAD committed before the failure must still show its effects afterward.

To do this without trusting anything held only in volatile memory at the moment of a crash, the recovery manager relies on a log — a sequential, append-only record of every change, written to stable storage independently of the data files themselves.

Log-Based Recovery & Write-Ahead Logging

Each log record captures: the transaction ID, the data item changed, its old value (for undo), and its new value (for redo). The write-ahead logging (WAL) protocol states the rule that makes this reliable:

A log record describing a change must be written to stable storage BEFORE the data page it describes is written to disk.

1. Log record written

Old value, new value, transaction ID — flushed to stable storage.

2. Data page written

The actual table/index page is updated on disk — only after step 1.

If the data page were written first and the crash happened before the log record, there would be no record of what changed or how to undo it. Writing the log first guarantees the only possible failure mode is a change the redo step can fix.

Undo and Redo

Undo

Reverses the effect of a transaction that had NOT committed at the time of the crash, using each log record's old value to restore the data item. Every uncommitted transaction active at the moment of the crash must be fully undone — Atomicity demands it leaves no partial trace.

Redo

Reapplies the effect of a transaction that HAD committed, using each log record's new value, to guarantee its change is actually present on disk — WAL only guarantees the log record was written before the crash, not that the data page write had completed.

Redo is idempotent on purpose: reapplying an already-durable write causes no harm — it just writes the same new value again.

Checkpointing

Naively, recovery would mean replaying the ENTIRE log from the first transaction the database ever ran — growing more expensive without bound. Checkpointing solves this.

Flush all dirty pages to disk

Every change committed before this instant is now durably on disk.

Write CHECKPOINT record to the log

Marks the boundary: nothing before this point needs to be replayed.

On a later crash, recovery starts HERE

Not from the beginning of the log — only from the last checkpoint forward.

Rollback and Rollforward

● Rollback

Undoing a transaction's changes — either because it aborted during normal operation, or because recovery determined it was still uncommitted at crash time. Mechanically identical to “undo.”

● Rollforward

Reapplying committed changes from a log to bring a database forward to a target point in time. Mechanically identical to “redo” — the term used after a media failure: restore the last backup, then rollforward through the log.

Worked Walkthrough: The Log Up to the Crash

Four transactions run against Accounts (A–E). A checkpoint occurs partway through, and the system crashes partway through the transactions that follow it.

#	Log record	Note
1	[T1 START]	
2	[T1, write A, old=100, new=200]	
3	[T1, write B, old=50, new=150]	
4	[T1 COMMIT]	T1 fully committed
5	[CHECKPOINT]	Dirty pages flushed; disk matches log through #4
6	[T2 START]	
7	[T2, write C, old=10, new=20]	
8	[T2 COMMIT]	T2 fully committed, after the checkpoint
9	[T3 START]	
10	[T3, write D, old=5, new=99]	
11	[T4 START]	
12	[T4, write E, old=1, new=2]	
13	[T4 COMMIT]	T4 fully committed, after the checkpoint
—	CRASH	System failure — RAM lost, disk intact. T3 never committed.

Step 1–2: Where Recovery Starts, and Classification

Because of the checkpoint at record 5, recovery does NOT need to look at records 1–4 at all — T1 committed before the checkpoint, so its changes are already guaranteed safe on disk. Recovery begins its analysis at record 5 and reads forward, classifying every transaction active at or after the checkpoint.

Transaction	Outcome by crash time	Action
T1	Committed before checkpoint	None needed — already durable
T2	Committed after checkpoint	Redo
T3	Never committed	Undo
T4	Committed after checkpoint	Redo

Redo → T2, T4 (committed after checkpoint). Undo → T3 (never committed). No action → T1 (committed before checkpoint).

Step 3–4: Redo Committed, Undo Uncommitted

Step 3 — Redo every committed transaction after the checkpoint

```
REDO: C = 20    (from record 7, T2)
REDO: E = 2     (from record 12, T4)
```

Step 4 — Undo every transaction that never committed

```
UNDO: D = 5     (from record 10, T3 —
restores D to its value before
T3 touched it)
```

Why T3 is undone even though it wrote to the log: having a log record for T3's write to D does not mean that write is entitled to survive — it only means the recovery manager has enough information to know what to undo. A log record's old-value field exists specifically so an uncommitted transaction's writes can be reversed with certainty.

The Result

Item	Final value	Why
A	200	From T1 — committed before the checkpoint, untouched
B	150	From T1 — committed before the checkpoint, untouched
C	20	From T2 — redone
D	5	From T3 — undone back to its original value
E	2	From T4 — redone

This is exactly the state the database would be in if T1, T2, and T4 had each run to completion and T3 had never run at all — precisely what Atomicity and Durability together require.

Key Takeaways

- Failures split into transaction failure, system crash (RAM lost, disk intact), and media failure (disk itself gone) — each needs a different response.
- The recovery manager guarantees Atomicity and Durability survive any failure, using a log written independently of the data files.
- Write-ahead logging (WAL) requires the log record to reach stable storage before the data page itself does.
- Undo reverses uncommitted transactions using old values; redo reapplies committed transactions' new values — redo is safely idempotent.
- Checkpointing bounds recovery to replay only from the last checkpoint forward — the worked trace redid T2 and T4, undid T3, left T1 untouched.